Vehicular Task Offloading Method Based on D3QN and Hierarchical Federated Learning

Rui Cao[a]

[a] School of Artificial Intelligence & Hebei Province Key Laboratory of Big Data Calculation, Hebei University of Technology, Tianjin, China

ARTICLE INFO

Keywords:
Task offloading
Federated learning
Internet of Vehicles

ABSTRACT

With the developments of the intelligent vehicles, the most significant contradiction is coming to the limited computing resource and the unlimited requirements. Task offloading has been proposed to overcome this. However, most of the exiting methods are based on complete resource information scenarios in a centralized way which is not up the real scenarios. Because, factors such as data communication delay, task delay constraints, and user privacy requirements will make it difficult to obtain global node information in a short time. This paper first formulates a dynamic optimization problem for delay and energy consumption based on multi-vehicle and multi-base station scenario, and designs a distributed offloading algorithm based on Dueling Double Deep Q-Network. The algorithm learns the resource distribution according to the user's historical feedback and makes a suitable offloading strategy for the current task. In order to solve the problem of unstable environment caused by vehicle movement, this paper combines the hierarchical federated learning mechanism to improve the learning efficiency and reduce the redundant interaction process. The simulation results show that our algorithm improves the optimization effect of delay and energy consumption by 5.06% and 6.25% respectively, and improves the task success rate by 7.32% compared with the latest method.

1. Introduction

The advancement of connected vehicle technology has spurred the creation of various types of applications, such as autonomous driving and virtual reality. However, these applications often require tasks to be completed within strict time constraints, which is often unfeasible due to the limited computing power of vehicles [1, 2]. In the past, delivering tasks to cloud servers for computation was a common approach. However, this method is no longer sustainable due to the exponential increase in data volume, leading to unpredictably long transmission delays and placing significant demands on bandwidth resources. Task offloading method based on MEC has been an effective approach to overcome these challenges by offloading tasks to nearby edge servers [3, 4]. This method can satisfy computing delay requirements while ensuring the reliability of results. Nevertheless, task offloading is still a challenging problem due to the following reasons:

(1) Uncertainty of edge servers' computational capabilities: Users often have several optional nodes for task offloading, but the computing power of these nodes is unknown to the user. If the information is obtained by means of communication, it will bring additional delay.

(2) Environmental dynamics: The time-varying nature of the channel, the mobility characteristics of the vehicle, and the heterogeneity of the task will make it difficult to make offloading decisions.

(3) Limitations of vehicle energy consumption: Low-carbon travel is the development trend of the automotive industry in the future. For new energy vehicles, limited battery capacity and endurance should also be considered in the algorithm. For emergency tasks, priority should be given to ensuring that tasks can be completed within the specified time. For secondary tasks, it is necessary to balance delay and energy consumption as much as possible.

The existing work rarely considers the complex characteristics of the Internet of Vehicles environment when making task offloading decisions, such as task urgency, vehicle speed, vehicle location and user travel demand. In order to simplify the problem, many schemes default that the resource information of edge nodes is known, and design algorithms on the known data to solve the task offloading problem, which is unrealistic in practical applications.

In order to solve these problems, this paper comprehensively considers various dynamic factors of the Internet of Vehicles, and uses deep reinforcement learning technology to propose a distributed task offloading algorithm based on D3QN, which learns the resource distribution characteristics of the environment through the continuous interaction process between the vehicle and the environment. In addition, we combine the hierarchical federated learning mechanism to accelerate the training efficiency of the algorithm and maximize the long-term utility through the knowledge sharing and collaborative training of vehicles in multiple regions without revealing personal privacy.

2. Related work

The difference between the Internet of Vehicles and other networks lies in the dynamic network topology brought by vehicle movement[5, 6]. Most of the current research focuses on the mobility of vehicles and the dynamics of resources. From the perspective of information dependence, the current research can be divided into task offloading methods based on complete information and task offloading methods under incomplete information.

*Corresponding author
ORCID(s):

The task offloading method based on complete information refers to the design of related algorithms to solve the task offloading problem under the premise that the node resource information is known, and there has been a lot of research work. Reference [7] studied the joint optimization of computation offloading and resource allocation in dynamic multi-user MEC systems, and proposed a reinforcement learning method based on value iteration. In Reference [8], Dai et al.considered the task offloading problem in the data-driven scenario. Based on the idea of A3C[9] and DQN, the author proposed an asynchronous ADQN algorithm for task offloading. The model uses asynchronous training to make the global model converge quickly. In Reference [10], the author considers the dynamic optimization requirements of different applications between service delay and result quality loss, and proposes an event-triggered dynamic task allocation framework based on linear programming optimization and binary particle swarm optimization. In order to protect the security threats in task offloading, Reference [11] proposed a security and cost-aware load computing strategy, and used a deep Q-network to find the optimal solution of the problem. Reference [12] studied the cloud-edge collaborative mobile computing offloading mechanism for industrial networks, and proposed a cloud-edge collaborative mobile computing offloading algorithm based on deep reinforcement learning to solve the optimization problem, so as to minimize the weighted sum of execution delay and energy consumption of all tasks of all users. In order to make full use of vehicle resources, Reference [13] divided the task into multiple small tasks, and realized distributed task offloading based on DQN algorithm. Reference [14] considered the computation offloading problem in a multi-user multi-edge server scenario. The author designs a decentralized computing strategy to achieve the lowest delay and energy loss. Reference [15] considered the problem of task migration caused by user mobility, and designs a distributed task migration algorithm based on multi-agent reinforcement learning. Reference [16] used neural networks to predict traffic flow to achieve load adjustment, and a task offloading and resource allocation algorithm based on game theory and Q-learning is designed.

The above methods are based on complete information conditions as the premise of task offloading. However, in real scenarios, due to large communication delays and user privacy requirements, it is difficult to obtain global resource information in a timely and accurate manner. A small number of scholars have considered this problem and proposed a task offloading method under incomplete information conditions. In Reference [17], the author considered how to make decisions in ultra-dense networking scenarios with time-varying channel conditions. The channel gain is used as the input of the algorithm, and a distributed algorithm based on Double DQN is designed to solve the task offloading strategy. In Reference [18], considering the problem that the dynamic environment makes it difficult to obtain the real-time information of the base station, a deep reinforcement learning offloading method based on historical actions and feedback is proposed. The proposed method does not require instantaneous channel state information or a prior knowledge of the computing power of the base station. Reference [19] considered the task offloading method in the scenario where unknown information and users coexist, and used the multi-armed slot machine algorithm to solve it. Although the above algorithm solves the problem of dependence on information, it is limited to the ideal MEC scene. The algorithm not only needs to be trained by frequent interaction with the environment, but also can only be used in a limited range. Reference [1] proposed a task offloading method based on federated reinforcement learning. This method realized experience sharing from the perspective of global aggregation and solve the problem of geographical scope limitation. However, this method ignores the utilization of potential spatial relationship. For example, the resource distribution in the adjacent area is similar, and the vehicles in the same area do not need to be repeatedly learned, so this method still can be improved.

In order to solve this problem, this paper proposes a task offloading method combining deep reinforcement learning and hierarchical federated learning, which improves the original method from both global and local perspectives. Compared with the existing methods, it can effectively reduce the delay and energy loss, and achieve long-term utility maximization.

3. System model

We assume that a number of small-cell base stations (SBSs) and macro-cell base stations (MBSs) are deployed in an area. Each base station is equipped with an edge server. The SBS is responsible for task offloading and the MBS is the manager of the region, as shown in Figure 1. Suppose that the set of SBS servers is $\mathcal{M} = \{1, 2, \dots M\}$, each server is defined as m and $m \in \mathcal{M}$. The characteristics of each server can be described by the tuple (f_m, b_m), which represents the computing power and wireless bandwidth of the edge server m, respectively. We discretize time into a set of equal-interval time slots, denoted as $\mathcal{T} = \{1, 2, \dots, T\}$, and each time slot $t \in \mathcal{T}$[20].

It is assumed that the vehicle u runs on the road at a speed of v_u and its position is (x_u^t, y_u^t) at time slot t. The vehicle carries a number of tasks and can choose to calculate its tasks locally or offload partial task to the edge server. However, due to the limitation of communication range, the vehicle can only choose the SBS within the signal coverage. The set of tasks is $\mathcal{R} = \{1, 2, \dots, R\}$. Each task $r \in \mathcal{R}$ and can only be processed in order. Task r has three characteristics $\left(D_r, C_r, \phi_r\right)$. D_r represents the data size of task r, C_r represents the required CPU cycles of task r, and ϕ_r is the maximum tolerable delay of task r. The offloading action corresponding to task r is defined as a_r. When the task r is offloaded to server m, there is $a_r = m$, and $a_r^m = 1$, otherwise $a_r^m = 0$. If the task is calculated locally, the variable $a_r = 0$ and $a_r^l = 1$.

The total delay of the task includes three parts: transmission time, computation time and additional time. If the

task is completed at the server m and the vehicle has left the coverage of the SBS, several relay MBSs are required to transmit the calculation results of the task, and additional processing time is required in this process. The total energy consumption of the task includes the transmission energy consumption sent to the base station and the local computing energy consumption. We model the local computing and edge computing respectively.

3.1. Local Computing

When the task r is calculated locally, assuming that f_{local} is the CPU frequency of the vehicle, the calculation time is:

$$T_{local} = \frac{C_r}{f_{local}} \tag{1}$$

According to [21, 22], the local computing energy consumption in this process is

$$E_{local} = \kappa {f_{local}}^2 C_r \tag{2}$$

Where κ is the effective switched capacitance depending on chip architecture of the device [23].

3.2. Edge computing

When the task r is offloaded to server m and the offloading ratio is θ_r, the transmission time can be defined as T_{trans}, which can be calculated as:

$$T_{trans} = \frac{\theta_r \cdot D_r}{R_{mu}(t)} \tag{3}$$

$R_{mu}(t)$ is the channel data rate between the vehicle u and m, which can be calculated as:

$$R_{mu}(t) = b_m \log_2 \left[1 + \frac{\rho_0 p_r}{\sigma^2 \left[d_m(t) \right]^\beta} \right] \tag{4}$$

Where ρ_0 is the channel power gain at this distance, β is the path-loss index, p_r is the transmit power, σ^2 is the power noise[24], and $d_m(t)$ is the distance between the vehicle and m at time t.

Define the computing resource allocated to the task r by server m as f_m^r, and the calculation time T_{cmp} can be calculated as:

$$T_{cmp} = \frac{\theta_r \cdot C_r}{f_m^r} \tag{5}$$

If the vehicle has driven out of the communication range of the m when the task calculation is completed, there is a variable $z_r = 1$, otherwise $z_r = 0$. At this time, the calculation results need to be returned by several MBSs as relay points. Assuming that the number of relay points is k, the additional communication time is:

$$T_{ext} = k \cdot \lambda \tag{6}$$

λ is the turnaround time of data in the MBS, and its value is a fixed constant. Since the calculation results of the task are often much smaller than the size of the original data, the delay of the calculated results transmitted back to the vehicle is not considered [25, 26].

The offloading delay of the task can be expressed as the maximum value of the local computing delay and the offloading delay:

$$T_{off} = \max \left(T_{trans} + T_{cmp} + z_r \cdot T_{ext}, \frac{C_r \cdot (1 - \theta_r)}{f_{local}} \right) \tag{7}$$

Offloading tasks to the edge server will bring energy loss. Energy consumption is related to transmission time and transmission power. Assuming that the vehicle is equipped with multiple transmitters with different powers, the transmit power of the selected transmitter is p_r, the energy consumed by task r can be calculated as:

$$E_{off} = p_r T_{trans} + \kappa \cdot f_{local}^2 \cdot C_r \cdot (1 - \theta_r) \tag{8}$$

Combining the above two cases, the total offloading delay of task r is:

$$T_r = \sum_{\forall m \in \mathcal{M}} a_r^m T_{off} + a_r^l T_{local} \tag{9}$$

The total energy consumption is:

$$E_r = \sum_{\forall m \in \mathcal{M}} a_r^m E_{off} + a_r^l E_{local} \tag{10}$$

Assuming that the residual energy of the vehicle is E_{res}, the energy consumption of task r can't exceed the residual energy, so the constraint condition can be expressed as:

$$E_r < E_{res} \tag{11}$$

The vehicle updates the remaining energy at the moment after each task offloading:

$$E_{res} \leftarrow E_{res} - E_r \tag{12}$$

4. Problem Formulation

Due to the high dynamic characteristics of the vehicular network, the impact of delay and energy consumption cannot be weighted simply. We need to adjust the weight according to the environmental characteristic.

Suppose that the vehicle's task to be calculated is r,and the maximum tolerable delay is ϕ_r. We use the vehicle speed and the maximum tolerable delay of the task to prioritize the task. The weight of delay can be calculated as:

$$W_r^l = \frac{\alpha}{\phi_r} \cdot v_u \tag{13}$$

Where α is a scaling factor to adjust the weight. Next, we define the weight of energy consumption. Taking the new energy vehicle as an example, we assume that the maximum mileage of the vehicle u under full power is L_u, the remaining distance from the destination is S_u, the expected power consumption is roughly expressed as E_p, and the power used for task offloading is E_{ext}, which can be calculated as:

$$E_p = \frac{S_u}{L_u} \tag{14}$$

$$E_{ext} = E_{res} - E_p \tag{15}$$

When E_{ext} is relatively sufficient, we use a constant N to weight the energy consumption. At this time, the delay is the main optimization target. When E_{ext} is less than the threshold H, we weight it in logarithmic form. At this time, the weight of energy consumption is increased rapidly. The weight of energy consumption can be calculated as:

$$W_r^e = \begin{cases} \omega_e \cdot \mathrm{N}, & E_{ext} > \mathrm{H} \\ \min\left(-\omega_e \cdot \ln\left(E_{ext}\right), P_{max}\right), & E_{ext} \leq \mathrm{H} \end{cases} \tag{16}$$

Where P_{max} is the upper limit of the value, ω_e is the weight coefficient, N is a constant, H is the energy consumption threshold and satisfies $\mathrm{H} = e^{-\mathrm{N}}$. Therefore, the optimization objective OV can be expressed as:

$$OV = \sum_{\forall r \in \mathcal{R}} W_r^l \times T_r + W_r^e \times E_r \tag{17}$$

The final optimization problem is defined as:

$$\begin{aligned} & \min_{a_r^l, a_r^m, \theta_r} \frac{OV}{|\mathcal{R}|} \\ s.t. \quad & C1: \quad E_r < E_{res}, \forall r \in \mathcal{R} \\ & C2: \quad a_r^l + \sum_{\forall m \in \mathcal{M}} a_r^m \leq 1, \forall r \in \mathcal{R} \\ & C3: \quad a_r^l \in \{0,1\}, \forall r \in \mathcal{R} \\ & C4: \quad a_r^m \in \{0,1\}, \forall r \in \mathcal{R} \\ & C5: \quad \theta_r \in \{\theta_1, \theta_2, \ldots, \theta_n\} \\ & C6: \quad T_r \leq \phi_r, \forall r \in \mathcal{R} \end{aligned} \tag{18}$$

The C1 is the energy constraint. The C2 is the offloading action constraint and the vehicle can only select one offloading action. The C3 and C4 constrain the range of the action, and C5 indicates that the offloading ratio can only take the optional discrete value to reduce the complexity of the problem.The constraint condition C6 indicates that the task must be completed within the specified time[27].

5. Algorithm Design

Traditional reinforcement learning algorithms such as Q-learning can solve the problem of task offloading under incomplete information, but these methods still face the following challenges: (1) Due to the limitation of vehicle mobility and communication range, the environment in which each agent is located is unstable, resulting in low learning efficiency. (2) The potential spatial relationship between agents cannot be effectively utilized. For example, the resource distribution in the adjacent area has a certain similarity, and the agent can avoid repeated learning processes through experience sharing. In order to solve these problems, we use the method of hierarchical federated learning to improve the reinforcement learning algorithm from the perspective of knowledge sharing to improve the effect of the algorithm.

This section first transforms the above optimization problem into a Markov Decision Process (MDP), then introduces the three basic components of MDP : state space, action space and reward function, as well as the action-value function and network update strategy in the algorithm. Finally, the definition of communication mask and the HF-D3QN algorithm proposed in this paper are given.

5.1. MDP Components

In order to avoid dependence on edge information, we learn environmental knowledge through historical offloading feedback of the vehicle. In order to guide the agent to choose a better decision, we use the offloading information and feedback results of the two latest historical tasks as a reference, and define the state space as $S_u(t)$, which consists of $S_u^1(t)$,$S_u^2(t)$,$S_u^3(t)$,$S_u^4(t)$. The specific definition is as follows:

(1) Historical task information:

$$\begin{aligned} S_u^1(t) = \{ & D_{r-2}, D_{r-1}, C_{r-2}, C_{r-1}, W_{r-2}^l, \\ & W_{r-1}^l, W_{r-2}^e, W_{r-1}^e \} \end{aligned} \tag{19}$$

(2) Current task Information:

$$S_u^2(t) = \{D_r, C_r, W_r^l, W_r^e\} \tag{20}$$

(3) Current vehicle information:

$$S_u^3(t) = \{x_u^t, y_u^t, S_u, L_u, E_{res}\} \tag{21}$$

(4) Historical action and feedback:

$$S_u^4(t) = \{a_{r-2}, a_{r-1}, T_{r-2}, T_{r-1}, E_{r-2}, E_{r-1}\} \tag{22}$$

The action space is defined as A, which includes four parts: local offloading action, edge offloading action, optional sending devices and offloading ratios. Assuming that the vehicle has δ different power sending devices and τ optional offloading ratios, then

$$\begin{aligned} A = \{local\} + \{a_{m1}, a_{m2}, \ldots, a_{m|\mathcal{M}|}\} \times \{dev_0, dev_1, \ldots, \\ dev_\delta\} \times \{ratio_0, ratio_1, \ldots, ratio_\tau\} \end{aligned} \tag{23}$$

The agent performs action $a_r \in A$ according to the state $S_u(t)$, and the environment will feedback a reward $R_u(t)$ to evaluate the action and guide the network to update the parameter weights. When the task is completed within the specified time, we use the negative number of the target value as the reward, otherwise return a smaller constant -G as the penalty, so there is

$$R_u(t) = \begin{cases} -(W_r^l \times T_r + W_r^e \times E_r), & \text{if success} \\ -\mathrm{G}, & \text{otherwise} \end{cases} \tag{24}$$

5.2. Action-value function

The reinforcement learning algorithm maximizes the long-term utility at present, that is, the cumulative utility from the current time slot to the end time slot is the largest[28]. The cumulative discount utility in time slot t can be expressed as $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^n R_{t+n}$, where γ is the discount factor, and the impact of future

time slot rewards on the present should be discounted. Due to the state transition probability and the location of the action selection, it is difficult to accurately obtain U_t. The randomness can be eliminated by Markov hypothesis and expectation, and the action-value function $Q^{\pi}(s_t, a_t)$ only related to the current state s_t and action a_t can be obtained. The calculation method is shown as:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{\pi}[U_t | S_t = s_t, A_t = a_t] \tag{25}$$

The traditional DQN algorithm uses a simple multi-layer neural network to simulate the action-value function. The value of each action is calculated directly, without considering the contribution of other actions. However, in the real environment, different actions have different contributions to the state. In order to better estimate the contribution of different actions to the state, Dueling DQN improves the network structure and proposes the state-value function and the advantage function[29, 30], as shown in Figure 2. At this time, the output value of the Q function can be obtained by the following two functions :

$$Q_{\pi}\left(s_t, a_t\right) = V\left(s_t\right) + A\left(s_t, a_t\right) - \frac{1}{|A|}\sum_{a\prime} A\left(s_t, a\prime\right) \tag{26}$$

$V(s_t)$ represents the state-value function, and $A(s_t, a_t)$ represents the advantage function, which are replaced by two different neural networks. By decomposing the Q-value function into a state-value function and an advantage function, Dueling DQN can better estimate the contribution of different actions to the state, improve learning efficiency, and achieve better results in reinforcement learning tasks in complex environments.

5.3. Optimal policy acquisition

It is assumed that the strategy that can make the reinforcement learning problem optimal is π^*.This strategy can make the individual 's utility always greater than other strategies, but in reality, this strategy is difficult to find. Therefore, the local optimal solution is generally selected as a relatively good solution. By comparing the action-value brought by the strategy, the current optimal action is selected. The definition of the optimal strategy is shown as:

$$\pi^*(s_t) = \underset{a_t \in A}{\operatorname{argmax}}\, Q^*(s_t, a_t) \tag{27}$$

5.4. Value-network update

The recursive relationship for the action-values at time step t and time step $t+1$ can be obtained from the Bellman equation, which is shown as:

$$Q_{\pi}(s_t, a_t) = R_t + \gamma \mathbb{E}_{\pi}[Q_{\pi}(s_{t+1}, a_{t+1})] \tag{28}$$

According to the above formula, the Q-value can be obtained in all cases, but for the case of large situation space, the traditional Q-learning method will produce huge time cost. The deep learning method uses neural network to approximate the Q-value and has high computational efficiency. Therefore, how to optimize the model parameter ω of the neural network is a core issue. Gradient descent is a method to update the model parameters, but it requires a comparison between the true value and the estimated value. We use $\gamma Q_{\pi}(s_{t+1}, a_{t+1})$ to approximate $\gamma \mathbb{E}_{\pi}[Q_{\pi}(s_{t+1}, a_{t+1})]$, and obtain the formula:

$$Q_{\pi}(s_t, a_t) - \gamma Q_{\pi}(s_{t+1}, a_{t+1}) = R_t \tag{29}$$

According to the greedy policy, when action a_t is taken in state s_t, and a new state s_{t+1} is obtained, selecting the maximum action-value for future rewards based on s_{t+1} yields the mean squared error loss function:

$$\frac{1}{q}\sum_{j=1}^{q}\left(R_j + \gamma \max_{a_{j+1}\in A} Q_{\pi}\left(s_{j+1}, a_{j+1}, \omega\right) - Q_{\pi}\left(s_j, a_j, \omega\right)\right)^2 \tag{30}$$

Among them, q is the number of samples sampled from the experience pool. By optimizing the gradient descent of the above formula, the parameters of the neural network can be gradually updated to make the estimation more accurate. However, due to the use of the max function, it is easy to lead to overestimation of future value. In order to solve this problem, the Double Q-learning algorithm takes the optimal action in the next state as the input, and obtains a more accurate prediction value by replacing the max function[31]. At this time, the modified loss function is:

$$\frac{1}{q}\sum_{j=1}^{q}\left(R_j + \gamma Q_{\pi}\left(s_{j+1}, \ \operatorname{argmax}_a\left(Q_{\pi}(s_{j+1}, a, \omega)\right), \omega\right) - Q_{\pi}(s_j, a_j, \omega)\right)^2 \tag{31}$$

5.5. Communication Mask

Because of the limited communication range and computing power of the base station, MBS will issue a mask to each vehicle according to the change of the environment, and the vehicle u cannot offload tasks to the base station outside the communication range. Therefore, the mask can be defined as $M_u(t) = \{m_1, m_2, \ldots, m_{|A|}\}$, where the mask corresponding to the optional action is 1, and the rest is 0. If a recent action a_{r-2} or a_{r-1} leads to the failure of task offloading, the mask $m_{a_{r-2}}$ or $m_{a_{r-1}}$ at the corresponding position is also 0. Assuming that the action-value function is $Q(t)$, so its output after the mask operation is:

$$Q'(t) = \{Q_1 m_1, Q_2 m_2, \ldots, Q_{|A|} m_{|A|}\} \tag{32}$$

5.6. HF-D3QN

Hierarchical Federated Learning is an extension of the traditional federated learning method combined with hierarchical structure[32, 33, 34]. It can effectively deal with the problems of uneven data distribution and high communication overhead, and improve the performance and efficiency of the system in large and small scale. Due to vehicle mobility and limited communication range, agents are always in an unstable environment, resulting in extremely low learning efficiency. In order to solve this problem, an offloading

algorithm HF-D3QN based on Dueling Double Deep Q-Network and hierarchical federated learning is proposed. As shown in Figure 3, the algorithm mainly includes three parts:

(1) Local training. Each vehicle will deploy a reinforcement learning model D3QN locally. The model is based on the DQN algorithm. The Dueling Q-network architecture is used to improve the original multi-layer neural network, and the Double Q-learning method is used to update the network parameters. When the vehicle generates a new task, the user inputs the corresponding state vector into the prediction network to obtain the corresponding action-value. The agent uses the ϵ-greedy strategy to select the action with the highest value, and the environment will give a reward value as feedback after performing the action. When the collected empirical data reaches a certain threshold, the algorithm uses random sampling and gradient descent methods to update the local prediction network parameters. When a certain number of iterations is met, the target network parameters are updated simultaneously.

(2) Global aggregation. In the 'edge-end' layer, the algorithm aggregates the local models of each vehicle, and uses the learning experience and advantages of different vehicles to obtain a more global and comprehensive model to speed up the model learning efficiency. Specifically, users upload the local network parameter weights and the last reward value to MBS. Each MBS is responsible for the global aggregation and distribution of all client network models in the region. Considering the different user contributions, MBS will aggregate and distribute the collected sub-models based on the size of the reward value uploaded by the user. Users can choose to download the latest global network parameters to update the local prediction network, and use this as a basis for incremental learning to improve the efficiency of model training.

(3) Partial aggregation. Due to the similarity of resource distribution in the adjacent area, in order to avoid repeated learning and reduce global communication overhead, in the 'end-to-end' layer, short-range communication technologies (such as DSRC[35]) can be used to perform partial aggregation directly between vehicles. The vehicle u can actively send a broadcast request with a radius of R to the neighbor vehicle set $\mathcal{U}\prime$, and merge into a cluster with vehicle i ($\forall i \in \mathcal{U}'$) that satisfy $\sqrt{(x_u^t - x_i^t)^2 + (y_u^t - y_i^t)^2} < \mathrm{I}$ and $|v_i - v_u| < \mathrm{J}$ in the region, where I is the maximum reliable communication distance and J is the maximum acceptable speed difference.

Considering the difference in vehicle performance and average relative distance, we define the following scoring function:

$$G(u) = -k_1 \frac{1}{|\mathcal{U}'|} \sum_{\forall i \in \mathcal{U}'} \sqrt{(x_u^t - x_i^t)^2 + (y_u^t - y_i^t)^2} + k_2 f_u \tag{33}$$

Where k_1 and k_2 are weighted coefficients, both greater than zero. Each vehicle can calculate the score of the current user according to $G(u)$ and share it with all nodes in the cluster. The node with the highest score will be elected as the cluster head, and the remaining vehicles will send the local model parameters to the cluster head node. After the cluster head node receives the model parameters of multiple vehicles, the average aggregation algorithm is used to quickly aggregate and distribute these parameters. The communication between vehicles has a lower delay, which can effectively improve the efficiency and real-time performance of model aggregation.

When the vehicle enters the MBS range for the first time, a partial aggregation will be triggered, followed by a trigger at a fixed time interval. For global aggregation, the algorithm uses a number threshold, which is triggered when the number of models participating in aggregation reaches the threshold. The specific value of the threshold can be set and adjusted according to the actual scene and the effect of the algorithm. In summary, the HF-D3QN algorithm process in this paper can be expressed as algorithm 1.

In addition, in order to make the model adapt to the environment changes (such as a node is unavailable or hardware changes) in a timely manner, the entire federated learning process is lifelong, and each client must regularly upload local network parameters to update the global network.

6. Performance Evaluation

In this section, we first introduce the simulation setup and baselines. Then we evaluate the performance of our algorithm through a large number of simulations.

6.1. Simulation Setup

We simulate a 2km × 2km area with 36 SBSs and 1 MBS. The computing power of the edge server obeys the uniform distribution of [5,15]GHz and each server allocates resources equally according to the number of tasks received. The turnaround time in the MBS is 50ms, the path-loss index $\beta = 4$, the channel bandwidth of the base station is set to 10 MHz, and the noise power $\sigma^2 = 2 \times 10^{-13}$W. The schematic diagram of the simulation environment is shown in Figure 4.

For the user's transmit device, the available transmit power is 150mW or 300mW. We define three levels of offloading ratios: 0.33, 0.66 and 1. The battery capacity E_{max} is 100% and the threshold H is 0.37. The input data size $D_r \in [0.3, 4]$Mbits and the required CPU cycles $C_r \in [2, 10]$GHz/task, the maximum tolerable delay $\phi_r \in [200, 1000]$ms. And the vehicle uses a random walk model[17].

In addition, we set the size of the experience pool to 2000, the batch size to 128, the discount factor to 0.9, the learning rate to 0.01, the exploration probability to 0.05, the update frequency of the target network parameters to 200. The number of user clients is 20, and their models are trained for 600 episodes in parallel. After each episode, the vehicle's speed, location, initial power, remaining mileage and other information are changed randomly to cover different vehicle states, and the global model (HF-D3QN) are updated and evaluated. The baseline algorithm includes greedy algorithm (Greedy), random offloading algorithm (Random), and local

Algorithm 1: HF-D3QN

input : Global network Q_g, current user u, prediction network Q_u, target network Q_u^t, neighbor set $\mathcal{U}'$, update threshold Th1, partial threshold Th2, global threshold Th3, MBS receiving set $\mathcal{G}$;

output: Local prediction network Q_u, local target network Q_u^t;

initialization;
$step = 0$;
$Q_u = Q_g$;
$startTime$ = time.time();
for i *in* $\mathcal{R}$ **do**
  $currentTime$ = time.time();
  $deltaTime = currentTime - startTime$;
  if $step == 0 || deltaTime >=$ Th2 **then**
    Broadcast aggregation request to the set $\mathcal{U}'$;
    Obtain cluster according to the relative distance and speed difference.;
    $Q_{part} = \frac{1}{|\mathcal{U}'|+1}\left(\sum_{\forall i \in \mathcal{U}'} Q_i + Q_u\right)$;
    $Q_u = Q_{part}$, $Q_u^t = Q_{part}$;
    resetStartTime($\mathcal{U}'$);
    $startTime = currentTime$;
  end
  The best action is obtained by Q_u;
  The user offloads the task;
  Get reward value R_u, step += 1;
  Update the local network Q_u;
  if $step$%Th1 == 0 **then**
    Upload the latest reward R_u and the local network Q_u;
    $Q_u^t = Q_u$;
  end
  if *Number of MBS receiving models* == Th3 **then**
    Trigger global aggregation and issue the model;
    $w_u = \frac{e^{R_u}}{\sum_{\forall i \in \mathcal{G}} e^{R_i}}$;
    $Q_g = \sum_{\forall u \in \mathcal{U}} w_u \cdot Q_u$;
    Update the local parameter $Q_u = Q_g$;
  end
  $s(t) = s(t+1)$;
end

computing algorithm (Local)[36, 37]. The greedy algorithm will choose to offload all tasks to the nearest edge server. The random offloading algorithm uses a completely random method to offload the task, and the local computing algorithm will process all tasks locally. Considering the heterogeneity of vehicles and tasks, we use weighted delay and energy consumption to measure the effectiveness of the algorithm.

6.2. Evaluation Results

We firstly combine the hierarchical federated learning mechanism with several deep reinforcement learning algorithms for discrete action space: DQN, Nature DQN, Double DQN, Dueling DQN, Dueling Double DQN to obtain five task offloading algorithms HF-DQN, HF-NaDQN, HF-DDQN, HF-DeulDQN and HF-D3QN. The corresponding training curve is in Figure 5.

In the simulation environment of this paper, the HF-DuelDQN algorithm based on Dueling DQN has faster convergence speed, but the HF-D3QN algorithm based on D3QN has better convergence effect. Due to the lack of the assistance of the target network, the HF-DQN algorithm performs worst in convergence speed and final convergence level, and it is difficult to converge to the optimal solution. The HF-NaDQN algorithm uses two independent Q-networks, one for selecting the current action and the other for evaluating the target Q-value. This design can reduce the overestimation of the target Q-value, so the optimization effect is better than HF-DQN.

In addition to comparing with several classical reinforcement learning algorithms, this paper applies several offloading algorithms in the existing literature to the scenarios of this paper, including ADQN [8], COA [17] and A3C-MEC [18]. The ADQN algorithm needs to obtain edge information in advance, which belongs to the complete information category algorithm, and the rest are incomplete information algorithms, which do not require environment information as input. The final experimental results are shown in Figure 6.

It can be seen that the HF-D3QN algorithm based on multi-vehicle cooperative training is significantly better than the COA algorithm based on single-vehicle local training in terms of convergence speed. The reason is that the limited communication range of the vehicle leads to the fact that the agent can only explore limited regional features, and the HF-D3QN algorithm can integrate the model parameters of multiple regional vehicles and use knowledge sharing to improve learning efficiency. Due to the limited optional actions, the performance of the A3C-MEC is poor and the optimization effect is basically the same as that of Greedy. The ADQN algorithm adopts the mechanism of asynchronous training, and has the base station information input as a reference, so it performs better in convergence speed and final effect, but the disadvantage is that it has a certain dependence on edge information, which reduces the reliability of the algorithm to a certain extent.

In order to analyze the timeliness of the algorithm, we compare the execution time of different algorithms in calculating the single task offloading decision. The results are shown in Figure 7.

Because the ADQN algorithm needs to obtain the edge information in advance, it has the longest execution time

among all the comparison algorithms. The HF-D3QN, A3C-MEC and COA algorithms do not need to obtain the edge information in advance, and only need the local calculation delay, so the algorithm delay is relatively low. Greedy and Random require neither environmental information nor complex matrix calculations, so the single execution time is significantly the shortest.

Then we choose COA, A3C-MEC algorithms and Greedy, Random, Local three baseline offloading strategies as comparison algorithms to observe their optimization effects on delay, energy consumption, and task success rate. Firstly, when the initial power of the vehicle changes, the delay and energy consumption of different algorithms are shown in Figure 8 and 9.

Figure 8 compares the delay optimization effect of the algorithm under different initial electric quantities. Due to the weak local computing ability of the vehicle, it takes a lot of time to use local computing, which leads to the worst delay optimization effect. The greedy algorithm is superior to the local and random algorithms in delay optimization, but inferior to A3C-MEC and COA. In general, HF-D3QN has the lowest task delay. Figure 9 compares the energy consumption of different algorithms. As the amount of electricity decreases, the energy consumption weight gradually increases, so the energy consumption of the algorithm is increasing. Due to the relatively small amount of data of the task, the greedy algorithm is used to offload the task to the nearest node. It will only bring a small amount of transmission energy consumption, so the energy consumption of the greedy algorithm is the lowest, but the nearest node is not necessarily the optimal node, the delay can not be guaranteed to be the lowest.

Figure 10 compares the task offloading success rate of the algorithm under different initial power. Because HF-D3QN adopts the strategy of task segmentation, the algorithm can select the appropriate task segmentation ratio according to the characteristics of environmental resources, and reduce the delay through edge-end parallel computing, so it has certain advantages in task success rate. Similarly, due to the limitation of the vehicle 's own computing power, the local computing strategy has the lowest success rate.

Since the change of vehicle speed will affect the task offloading delay and delay weight, we tested the delay and energy consumption comparison under different vehicle speeds. The experimental results are shown in Fig.11 and Fig.12.

When the vehicle speed is faster, the weight of the delay also increases, so the curve is on the rise as a whole, but the energy consumption is not affected by the vehicle speed, and the curve is roughly parallel. The delay of COA and A3C-MEC is between that of Greedy and our method, and the energy consumption of Greedy remains the lowest.

The success rate of the task at different speeds is shown in Fig.13. It can be seen that the offloading success rate of the method in this paper is still the highest at different speeds. Therefore, considering the three indicators of delay, energy consumption and task success rate, the HF-D3QN algorithm is significantly better than the other algorithms.

When the vehicle leaves the current area and enters a new MBS area, the algorithm can use the global model trained in the new MBS as the initial parameters of the local network to achieve the effect of 'hot start', as shown in Figure 14. However, other distributed algorithms suffer from a sharp decline in their effectiveness due to knowledge expiration. These algorithms do not consider experience sharing mechanisms and therefore require continuous retraining. Even worse, due to the influence of previous parameters, the model may take longer to converge.

7. Contribution

In this paper, we propose a vehicular task offloading method with incomplete information. We first construct a multi-vehicle multi-base station scenario based on mobile edge computing, and then implement a dynamic weighting method for delay and energy consumption based on a variety of objective factors. Finally, we combine deep reinforcement learning and hierarchical federated learning to propose a distributed task offloading algorithm HF-D3QN. We have carried out a large number of simulation experiments to verify the effect of our algorithm. The results show that the algorithm can achieve fast convergence and achieve the best optimization effect under different power and speed conditions.

Acknowledgement

This work was supported in part by the Hebei Province Innovation Capability Enhancement Plan Project under Grant 22567603H, in part by S&T Program of Hebei under Grant 20310801D.

References

[1] Cao, R., Song, Z., Niu, B., Gu, J., Li, C., 2023. Distributed task offloading method based on federated reinforcement learning in vehicular networks with incomplete information, in: International Conference on Algorithms and Architectures for Parallel Processing, Springer.

[2] Liu, J., Ahmed, M., Mirza, M.A., Khan, W.U., Xu, D., Li, J., Aziz, A., Han, Z., 2022. Rl/drl meets vehicular task offloading using edge and vehicular cloudlet: A survey. IEEE Internet of Things Journal 9, 8315–8338.

[3] Li, Z., Qi, W., Yifan, C., Guoqi, X., Li, R., 2021a. A survey on task offloading research in vehicular edge computing. Chinese Journal of Computers 44, 963–982.

[4] Li, W., Liu, D., Chen, K., Li, K., Qi, H., 2021b. Hone: Mitigating stragglers in distributed stream processing with tuple scheduling. IEEE Transactions on Parallel and Distributed Systems 32.

[5] Panigrahy, S.K., Emany, H., 2023. A survey and tutorial on network optimization for intelligent transport system using the internet of vehicles. Sensors 23, 555.

[6] Hou, Y., Wei, Z., Zhang, R., Cheng, X., Yang, L., 2023. Hierarchical task offloading for vehicular fog computing based on multi-agent deep reinforcement learning. IEEE Transactions on Wireless Communications .

[7] Zhou, H., Jiang, K., Liu, X., Li, X., Leung, V.C., 2021. Deep reinforcement learning for energy-efficient computation offloading in

mobile-edge computing. IEEE Internet of Things Journal 9, 1517–1530.

[8] Dai, P., Hu, K., Wu, X., Xing, H., Yu, Z., 2021. Asynchronous deep reinforcement learning for data-driven task offloading in mec-empowered vehicular networks, in: IEEE INFOCOM 2021-IEEE Conference on Computer Communications, IEEE. pp. 1–10.

[9] Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: International conference on machine learning, PMLR. pp. 1928–1937.

[10] Zhu, C., Tao, J., Pastor, G., Xiao, Y., Ji, Y., Zhou, Q., Li, Y., Ylä-Jääski, A., 2018. Folo: Latency and quality optimized task allocation in vehicular fog computing. IEEE Internet of Things Journal 6, 4150–4161.

[11] Huang, B., Li, Y., Li, Z., Pan, L., Wang, S., Xu, Y., Hu, H., 2019. Security and cost-aware computation offloading via deep reinforcement learning in mobile edge computing. Wireless Communications and Mobile Computing 2019, 1–20.

[12] Chen, S., Chen, J., Miao, Y., Wang, Q., Zhao, C., 2022. Deep reinforcement learning-based cloud-edge collaborative mobile computation offloading in industrial networks. IEEE Transactions on Signal and Information Processing over Networks 8, 364–375.

[13] Chen, C., Zhang, Y., Wang, Z., Wan, S., Pei, Q., 2021. Distributed computation offloading method based on deep reinforcement learning in icv. Applied Soft Computing 103, 107108.

[14] Ding, Y., Liu, C., Zhou, X., Liu, Z., Tang, Z., 2019. A code-oriented partitioning computation offloading strategy for multiple users and multiple mobile edge computing servers. IEEE Transactions on Industrial Informatics 16, 4800–4810.

[15] Liu, C., Tang, F., Hu, Y., Li, K., Tang, Z., Li, K., 2020. Distributed task migration optimization in mec by extending multi-agent deep reinforcement learning approach. IEEE Transactions on Parallel and Distributed Systems 32, 1603–1614.

[16] Xu, X., Jiang, Q., Zhang, P., Cao, X., Khosravi, M.R., Alex, L.T., Qi, L., Dou, W., 2022. Game theory for distributed iov task offloading with fuzzy neural network in edge computing. IEEE Transactions on Fuzzy Systems 30, 4593–4604.

[17] Zhang, Y., Liu, T., Zhu, Y., Yang, Y., 2020. A deep reinforcement learning approach for online computation offloading in mobile edge computing, in: 2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS), IEEE. pp. 1–10.

[18] Lin, Z., Gu, B., Zhang, X., Yi, D., Han, Y., 2022. Online task offloading in udn: A deep reinforcement learning approach with incomplete information, in: 2022 IEEE Wireless Communications and Networking Conference (WCNC), IEEE. pp. 1236–1241.

[19] Wang, X., Ye, J., Lui, J.C., 2022. Decentralized task offloading in edge computing: a multi-user multi-armed bandit approach, in: IEEE INFOCOM 2022-IEEE Conference on Computer Communications, IEEE. pp. 1199–1208.

[20] Li, W., Yuan, X., Qu, W., Qi, H., Zhou, X., Chen, S., Xu, R., 2020. Efficient coflow transmission for distributed stream processing, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications, IEEE. pp. 1319–1328.

[21] Wu, C., Huang, Z., Zou, Y., 2022. Delay constrained hybrid task offloading of internet of vehicle: A deep reinforcement learning method. IEEE Access 10, 102778–102788.

[22] Li, W., Yuan, X., Li, K., Qi, H., Zhou, X., Xu, R., 2020. Endpoint-flexible coflow scheduling across geo-distributed datacenters. IEEE Transactions on Parallel and Distributed Systems 31, 2466–2481.

[23] Burd, T.D., Brodersen, R.W., 1996. Processor design for portable systems. Journal of VLSI signal processing systems for signal,image and video technology .

[24] Tang, D., Zhang, X., Li, M., Tao, X., 2020. Adaptive inference reinforcement learning for task offloading in vehicular edge computing systems, in: 2020 IEEE International Conference on Communications Workshops (ICC Workshops), IEEE. pp. 1–6.

[25] Klaimi, J., Senouci, S.M., Messous, M.A., 2018. Theoretical game approach for mobile users resource management in a vehicular fog computing environment, in: 2018 14th International Wireless Communications & Mobile Computing Conference (IWCMC), IEEE. pp. 452–457.

[26] Li, M., Gao, J., Zhang, N., Zhao, L., Shen, X., 2020. Collaborative computing in vehicular networks: a deep reinforcement learning approach, in: ICC 2020-2020 IEEE International Conference on Communications (ICC), IEEE. pp. 1–6.

[27] Xu, R., Li, W., Li, K., Zhou, X., Qi, H., 2020. Scheduling mix-coflows in datacenter networks. IEEE Transactions on Network and Service Management 18, 2002–2015.

[28] Wang, B., Liu, L., Wang, J., 2023. Deep reinforcement learning based on actor-critic for task offloading in vehicle edge computing, in: 2023 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), IEEE. pp. 1–6.

[29] Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., Freitas, N., 2016. Dueling network architectures for deep reinforcement learning, in: International conference on machine learning, PMLR. pp. 1995–2003.

[30] Ji, Y., Wang, Y., Zhao, H., Gui, G., Gacanin, H., Sari, H., Adachi, F., 2023. Multi-agent reinforcement learning resources allocation method using dueling double deep q-network in vehicular networks. IEEE Transactions on Vehicular Technology .

[31] Van Hasselt, H., Guez, A., Silver, D., 2016. Deep reinforcement learning with double q-learning, in: Proceedings of the AAAI conference on artificial intelligence.

[32] Li, T., Sahu, A.K., Talwalkar, A., Smith, V., 2020. Federated learning: Challenges, methods, and future directions. IEEE signal processing magazine 37, 50–60.

[33] Liu, L., Zhang, J., Song, S., Letaief, K.B., 2020. Client-edge-cloud hierarchical federated learning, in: ICC 2020-2020 IEEE International Conference on Communications (ICC), IEEE. pp. 1–6.

[34] Lim, W.Y.B., Ng, J.S., Xiong, Z., Niyato, D., Miao, C., Kim, D.I., 2021. Dynamic edge association and resource allocation in self-organizing hierarchical federated learning networks. IEEE Journal on Selected Areas in Communications 39, 3640–3653.

[35] Alrubaye, J.S., Shahgholi Ghahfarokhi, B., 2023. Resource-aware dbscan-based re-clustering in hybrid c-v2x/dsrc vehicular networks. Plos one 18, e0293662.

[36] Liu, Y., Yu, H., Xie, S., Zhang, Y., 2019. Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks. IEEE Transactions on Vehicular Technology 68, 11158–11168.

[37] Nath, S., Wu, J., 2020. Deep reinforcement learning for dynamic computation offloading and resource allocation in cache-assisted mobile edge computing systems. Intelligent and Converged Networks 1, 181–198.

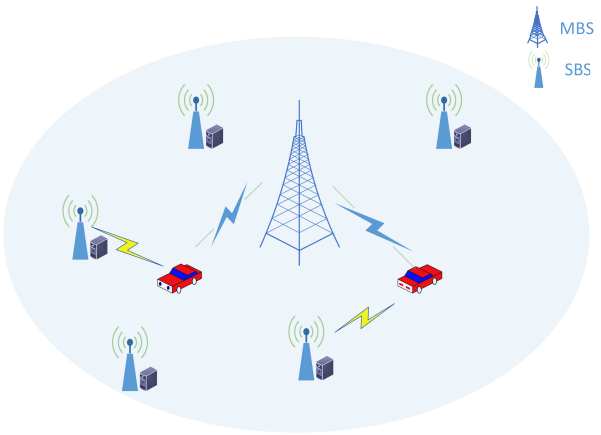

Fig. 1. Illustration of computation offloading scenario

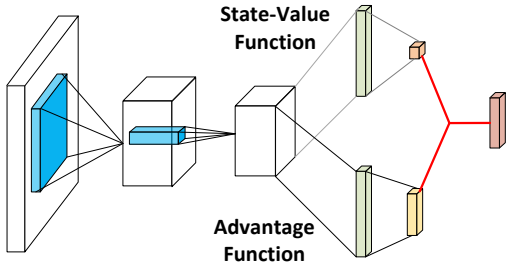

Fig. 2. Dueling Q-Network Structure.

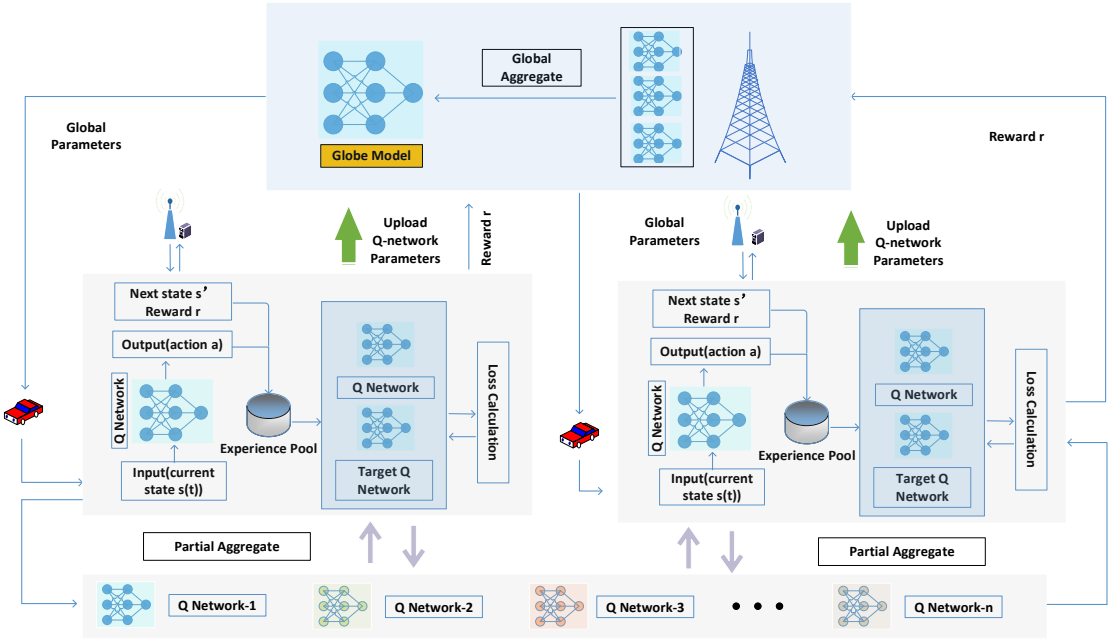

Fig. 3. HF-D3QN procedure.

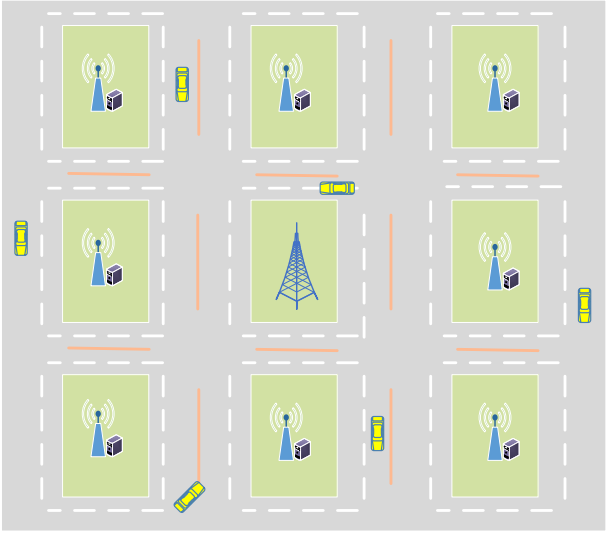

Fig. 4. Simplified simulation environment.

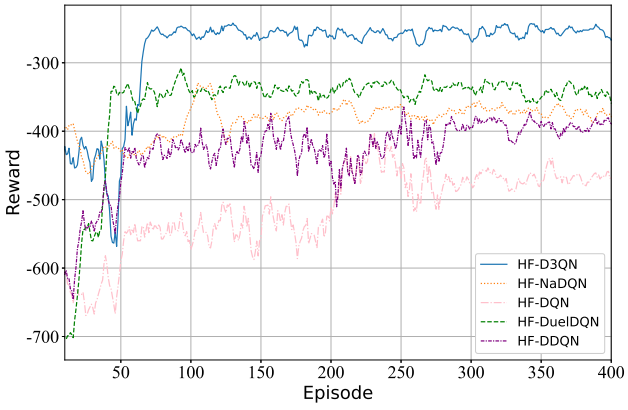

Fig. 5. Training curve of different combination.

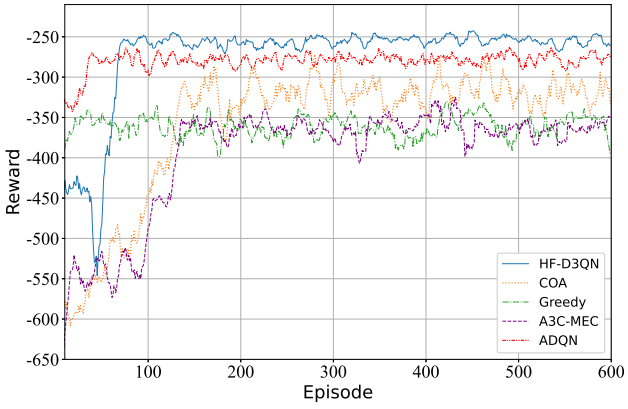

Fig. 6. Training curve of different algorithms.

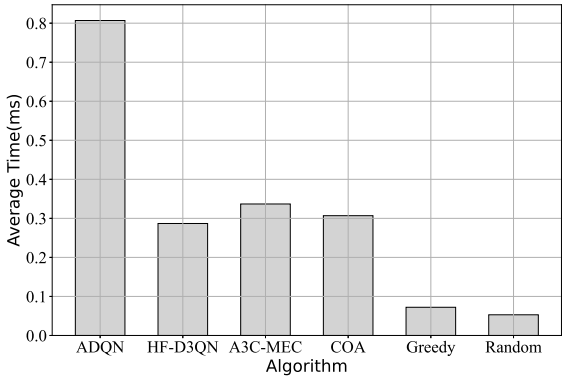

Fig. 7. Algorithm running time.

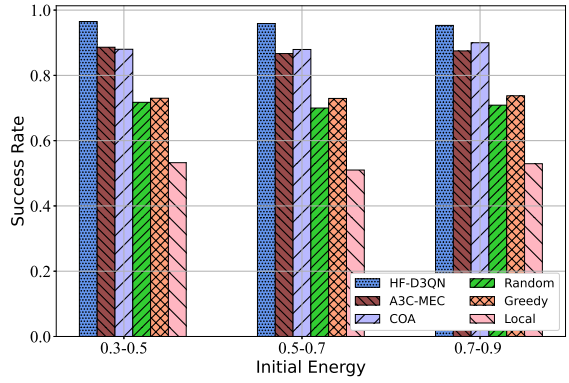

Fig. 10. Comparison of task success rate under different energy.

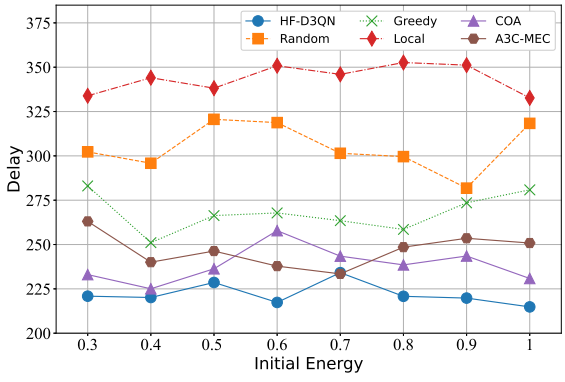

Fig. 8. Comparison of delay under different energy.

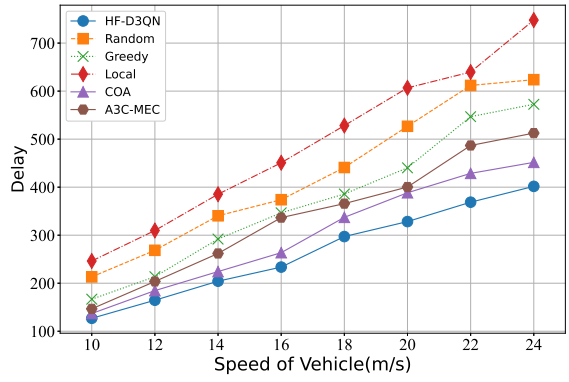

Fig. 11. Comparison of delay at different speeds.

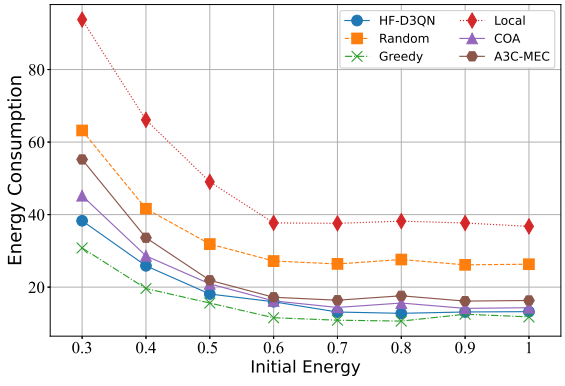

Fig. 9. Comparison of energy consumption under different energy.

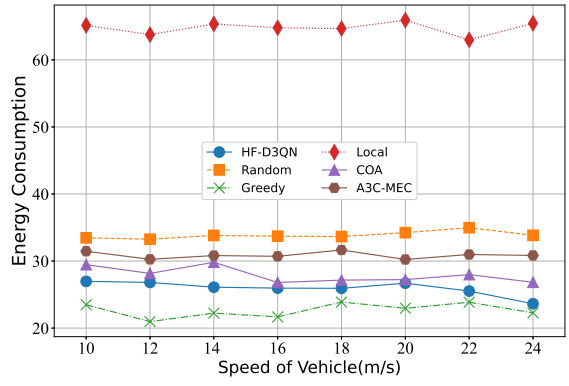

Fig. 12. Comparison of energy consumption at different speeds.

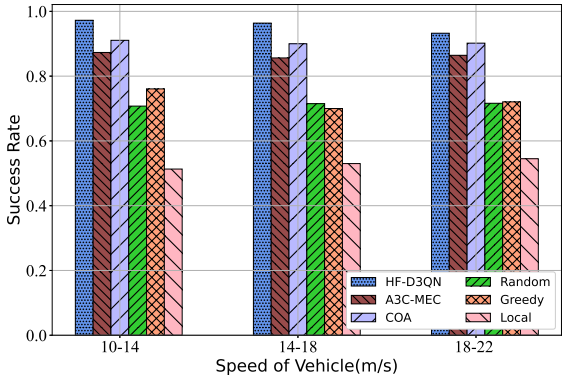

Fig. 13. Comparison of task success rates at different speeds.

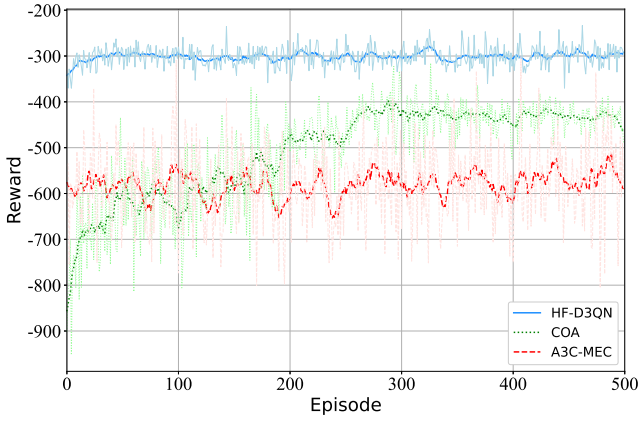

Fig. 14. Algorithm comparison at different environment.